

pqc-hsm-fpga 独立评审报告

评审对象：zu3eg-fpga-crypto @ 96f9b55 · 只读分析

- [一、总体评估](#)
 - [1.1 这个项目现在处于什么水平](#)
 - [1.2 核心价值在哪](#)
 - [1.3 离”真正可用的密码机”差什么](#)
- [二、问题与 bug](#)
 - [2.1 硬伤](#)
 - [2.2 隐患](#)
 - [2.3 其它值得记录的问题（优化级）](#)
 - [2.4 测试盲区](#)
- [三、可优化点](#)
 - [3.1 性能：结论是”赢面在时钟不在架构”，且这个结论是可信的](#)
 - [3.2 资源：约 1566 行 RTL 在产品形态下不可达](#)
 - [3.3 时序：余量是被管理的，但提频的路比看上去远](#)
 - [3.4 架构](#)
- [四、安全收口：假设 JTAG 已到位](#)
 - [4.1 先纠正一个前提：BBRAM 不等于信任根](#)
 - [4.2 事实基线（已核实）](#)
 - [4.3 关键判断：JTAG 的头号价值是”可恢复”，不是 BBRAM 本身](#)
 - [4.4 方案（按依赖顺序）](#)
 - [4.5 JTAG 到位后仍然做不到的](#)
- [五、优先级路线图](#)
 - [P0 — 立刻做（数小时到一天、风险极低）](#)
 - [P1 — 本周内（JTAG 相关的前两步 + 明确的隐患）](#)
 - [P2 — 本月（结构性问题）](#)
 - [P3 — 视产品决策而定](#)
- [附注：关于本报告的置信度](#)

本报告是一次**独立的批判性评审**，目标是把问题找出来，不是给项目背书。所有结论均引用具体文件与行号；凡是”评审员说”而我没能亲自复核的，都在文中标注了置信度。评审只读源码与仓库内的真板日志，**未**改动任何 RTL、未连板、未烧 eFUSE。

评审基准：`zu3eg-fpga-crypto` 分支 HEAD `96f9b55`。涉及性能的部分会明确区分本分支与 `zu3eg-fpga-perf` 分支（`c835f02`）。

一、总体评估

1.1 这个项目现在处于什么水平

它是一个做得相当认真的密码学硬件原型，工程素养明显高于同类自研项目的均线，但它现在还不是一台密码机，而是“一台密码机的硬件核心 + 一套围绕它的证据体系”。

先说值得肯定的，因为这决定了后面的问题该怎么定性——是“地基有问题”还是“地基不错但楼没盖完”。这个项目属于后者。

三件事做得是真好：

第一，密码边界是结构性的，不是许可性的。`key_vault` 的核心不变量——密钥寄存器到 AXI 读数据之间根本没有导线——我逐行核实成立：`key_vault.v:75-95` 的 `keys[]` 只出现在 `use_mux` 里，`key_vault_axi.v:264-289` 与 `sym_axi.v:280-296` 的读多路选择器只含元数据，`A_KEY_IN` 读恒返回 0 (`key_vault_axi.v:279`)。这与“加一个权限判断挡住读”是完全不同量级的保证：门控要求每一处副作用都记得判那个信号，漏判一处就是一个安静的洞；没有路径则没有可漏判的东西。`axi4lite_firewall` 同理，把策略做成编译期 parameter 而非运行时寄存器，从根上消掉了“谁能改策略”这个无限套娃问题。这一层是整个项目最值钱的地方，也是唯一一处我认为已经达到产品水准的设计。

第二，证据纪律是罕见的。失败被保留下来了。`board/logs/RESULT_restart.txt` 里躺着一条没有通过的重启熵测试（列 0 熵为 -0.000000 ，判据不通过），`RESULT_audit.txt` 里六条 SKIP 每条都写明为什么跳过。历史提交 `774935f` 记录了一次自我否定：按假设 $H=0.5$ 算出的 APT 阈值触发概率是 5.5×10^{-52} ，“这个检测从来就是死的，此前每一条‘无 APT 告警’的记录都不值钱”。一个愿意在自己的日志里写下这种话的项目，它给出的其它数字可信度会高很多。

第三，“只在真硅上现形”的坑被系统性地转化成了自动断言。`impl_bitstream.tcl` 把 fan 落点、`SYSMONE4` `SIM_DEVICE`、PS 时钟/复位驱动、WHS 地板、环振存活都做成综合后立即中止的检查。保持时序那一段尤其讲究：三份 bitstream 的 WHS 是 $+0.001/+0.010/+0.013$ ns，全为正、全被 Vivado 签核、全都贴着零，而保持违例不能靠降频解决。用 `set_clock_uncertainty -hold 0.100` 把要求提高、让布线器去满足，有效余量从 0.010 提到 0.110 ns。这是把“被观察的余量”变成“被管理的余量”，是成熟做法。

1.2 核心价值在哪

不在性能，在于它把“密钥不出硬件”这句话做成了可以被第三方检验的电路事实，并且把检验过程本身留成了证据。

这一点需要对照性能数据才能看清。`zu3eg-fpga-perf` 分支的实测结论是：A53 软件比 FPGA 快 4.33 倍（软件 1422 μ s vs FPGA-DMA 6164 μ s，九项合计）。更关键的是把搬运开销全部归零、只算纯核时间，FPGA 仍慢 3.3—4.5 倍。所以这条路的价值从来不在吞吐表里——它给的是另一组东西：密钥不出片、AxPROT 门控可反证、tamper 一根线全清、可独立于 PS 审计。

项目自己对这一点是清醒的（README:161-163 明写“这是一个安全原型，不是一个快的原型”），这份清醒本身就是价值的一部分。

1.3 离”真正可用的密码机”差什么

按距离由近及远：

缺口	性质	说明
信任根是空的	结构性	KDR 是编译期常量 (<code>src/crypto/kdr.c:11</code>)； BOOT.BIN 未签名 (<code>bootgen -w</code>)； PL bitstream 运行时从 SD 卡加载， root 可换。边界本身可被替换，则边界内的一切保证都是条件性的
主机栈从未驱动过真 PL	结构性	<code>ARCHITECTURE.md:234-238</code> 自己承认。PKCS#11 前端全程走 liboqs 软件后端；只有 <code>service/</code> 那条 SDF 路真的碰过硬件
私钥仍出硬件	已知	ML-KEM KeyGen 经 AXI 返回 <code>ek dk</code>
算法不全	已知	无 SM2；ML-DSA 只有 655 行算子，未串成核
物理防护未通电	已知	<code>tamper</code> 在顶层恒接 0
条件自检缺失	已知	无密钥对一致性检验、无连续 RNG 检验

一句话概括距离：密码学部分（算法正确性、边界结构、熵源质量）已经接近可用；信任链、密钥全生命周期管理、以及”文档能不能照着用”这三件事离可用还有明显距离。其中第三件最容易被低估，见 §2.1 的 H1。

二、问题与 bug

分级口径：硬伤 = 破坏了项目自己声明的核心不变量，或会让使用者受到实际损害；隐患 = 条件触发、当前接线下未暴露、或依赖尚未验证的前提；优化 = 健壮性与工程质量。

2.1 硬伤

H1 —— 文档与 RTL 倒挂：照 README + REGISTERS.md 操作会 panic 板子

这是本次评审发现的最要紧的一条，因为它会直接伤到使用者。

HEAD 的 RTL 是”送检口径”：四个功能从机全部硬编码 `SECURE_ONLY(1)` (`zu3eg_hsm_top.v:309, 327, 353, 371`)，顶层注释 `:40` 写得很清楚“【SECURE_ONLY：四个功能从机全都是 1 —— 送检口径】”。槽 4 的金丝雀现在在是“同参数下的同类项” (`:67`)，风扇槽 5 保持 0。

但三份面向使用者的文档描述的是已被替换掉的上一版位流：

- `README.md:47-49` 的架构图只给槽 4 标注 `SECURE_ONLY=1`；

- `README.md:56-58`：「槽 4 是密钥仓的第二个实例，只在 `SECURE_ONLY=1` 这一点上不同」——在 HEAD 上这句话已经不成立，因为四个功能从机都是 1；
- `SECURITY.md:88`：「同一个普通世界成功读到了一个 `SECURE_ONLY=0` 的核」——HEAD 上没有 `SECURE_ONLY=0` 的密码核，只有风扇是 0；
- `REGISTERS.md` 整份寄存器契约默认普通世界可达。

真正的危害在这里：顶层 `:63-65` 有一条警告，`docs/` 里一个字都没有：

⚠ 反向只读、不写。普通世界写一个被拒的地址，DECERR 是 posted 的，在 aarch64 上以 SError 回来——内核只能 panic。这条代价是一次断电。会写的旧程序（hsm_hwtest / hsm_kem3）**不能**对着这一版 bitstream 跑。

于是完整的失效场景是：使用者照 `README.md` 的 Quick start 出一份 bitstream → 照 `REGISTERS.md` 写驱动或直接跑仓库自带的板上程序 → 第一次写寄存器就 SError → 内核 panic → **必须断电**。而这块板子的启动实验风险很高（multiboot 由 POR 清零，坏镜像救不回来）。

严重性：硬伤。不是因为逻辑错了，而是因为文档会把使用者引到一个有代价的坑里，且该坑的警告只存在于一个使用者不会去读的 RTL 文件注释里。

建议：1. 立刻把这条 SError 警告提到 `README.md` 与 `REGISTERS.md` 顶部；2. 修正三处 `SECURE_ONLY` 描述，明确“当前位流 = 全 1，普通世界零可达”；3. `SECURITY.md:84-88` 的双向证据要注明它来自哪一版位流——证据本身是有效的，但必须说清是在哪个配置下取得的；4. 长远：把 `SECURE_ONLY` 做成构建开关（`SECURE_ONLY_FUNCTIONAL`），让“给 Linux 直连跑 KAT 的开发位流”和“送检位流”是同一份 RTL 的两个配置，而不是靠改源码切换——现在的状态是改过去就回不来，文档必然滞后。

H2 —— OP-TEE TA 的 `CMD_UNWRAP` 是私钥解密预言机

`tee/ta/pqchsm_ta.c:3-4` 的文件头声明：「安全边界：私钥明文不出 TA」。这条承诺被同一个文件里的另一个命令推翻了。

我逐行核实的链条：

- 私钥 blob 由 `keygen_finish` 封装：`ta_wrap_seal(g_kek, NULL, 0, sk, ...)`（`:246`）——用全局 `g_kek`，AAD 为空；
- `cmd_unwrap` 解封装：`ta_wrap_open(g_kek, params[0]..., params[1]...)`（`:206`）——同一个 `g_kek`，AAD 由调用方指定；
- `ta_wrap_open` 的全部校验是 magic / version / alg / `hdr_aad_len == aad_len` / 长度自洽（`ta_wrap.c:247-260`）。没有 blob 类型字段。

于是两类 blob 在密码学上**完全不可区分**。攻击路径（普通世界应用即可，无需 root，无需内核权限）：

1. `CMD_KEK_SET(salt)` —— salt 是密钥库头里的公开值，任何人可得；
2. `CMD_UNWRAP(aad = 空, blob = 某个私钥 blob)` —— 私钥 blob 按设计就存在普通世界（TA 把它交给调用方保管）；
3. `hdr_aad_len(0) == aad_len(0)` 通过，GCM tag 用同一 KEK 验证通过，TA 把私钥明文写回普通世界缓冲区。

KDR 的设备唯一性只挡跨设备盗用 blob，挡不住本机这个调用方——而“防普通世界应用”恰恰是该文件第 8-10 行声称要防的那一档。封装了私钥，却又提供一个用同一把钥匙的通用解封装命令，包裹本身就失去了意义。

严重性：硬伤。在“REE 内核可信”这个 TA 自己设定的前提下依然可被触发。

建议（两条择一，推荐第一条）：1. **域分隔**：私钥封装改用独立 KEK（不同 label，如 `ppc-hsm/privkey-kek`），只在 SIGN/DECAPS 内部使用；`CMD_WRAP` / `CMD_UNWRAP` 只用 storage KEK。这样 UNWRAP 拿 storage KEK 打不开私钥 blob，tag 直接失配。改动小、无协议变更。2. 在 PWRP 头里加一个进 AAD 的 `blob_type` 字段，`cmd_unwrap` 拒绝 `type == privkey`。

附带同源问题：`CMD_WRAP` 也是 storage KEK 下的加密预言机，可伪造任意“私钥 blob”。若上层把“能解开的 blob”当作 TA 来源凭证，需一并堵。

范围说明：`tee/` 在 README 里被标注为“separate line of work”，根目录 `SECURITY.md` 并未对 TA 作出承诺。所以这条不算“项目对外撒谎”，但它违反了 TA 文件自身写下的不变量，且该 TA 是“私钥运算移植到安全世界”这条线的全部成果，所以定性为硬伤。

H3 —— keystore 无防回滚，PIN 锁定形同虚设

`hsm_keystore_load`（`src/store/keystore.c:243`）的校验只有：magic、`HSM_KEYSTORE_VERSION`（恒为 1）、槽位数、全文件 MAC。没有任何单调计数器或 epoch。全文件 MAC 用同设备 KDR 复算，所以一份旧的、当初合法的 `keystore.bin` 能完美通过全部校验。

失效场景：– PIN 失败计数确实持久化并进了 MAC（`slot.c:596` 注释“断电重置绕不过去”），而 `p11_module.c:882` 每次 `C_Login` 尝试后都落盘。攻击者：拷走 `keystore.bin` → 试 2 次 → 还原旧文件（计数归零）→ 无限重试。`HSM_PIN_MAX_FAILS=3` 完全失效。– zeroize/destroy 之后落盘，还原 zeroize 之前的文件 → 已“销毁”的密钥复活。

这条即使换上真实 KDR 也不会自动消失：真 KDR 让 MAC 无法伪造，但不阻止回放一份自己曾经签发过的旧快照。防回滚需要外部单调锚（eFUSE 计数器 / RPMB / 安全存储版本号），纯文件系统方案天生给不了。

值得对照的是：审计日志有外部锚点机制兜底（`src/audit/anchor.c:267`），keystore 却没有对应设计。同一个仓库里两套持久化，一套想到了，一套没有。

严重性：硬伤（针对“防离线爆破”这一声明）。**建议**：落盘时写入来自防回滚存储的单调 epoch 并纳入 MAC；在拿到该能力之前，`SECURITY.md` 必须写明“防爆破与防回滚依赖底层存储介质，文件原型不具备”。

H4 —— KDR 桩的爆炸半径被低估了

`src/crypto/kdr.c:11-16` 是一个写死在二进制里的 32 字节常量，字面量本身就是“PQC-HSM STUB KDR -- NOT SECRET!!”。派生链上挂着 存储 KEK（`wrap.c:24`）、文件密钥（`keystore.c:77`）、元数据密钥（`meta.c:60`）。

这一条项目自己已经声明（`README.md:156-159`、`SECURITY.md:151`），所以不算隐瞒。但声明的措辞低估了后果。`kdr.c:8` 的注释说“在当前阶段它保护的只是测试数据”，而实际后果是：

- 任何人拿到 `keystore.bin`（无需拿到设备）即可复算 KEK，解出每个槽位的秘密载荷——包括私钥与种子，明文可恢复；
- `slot.c:192-195` 那句“攻击者拿到密钥库文件也无法离线枚举 PIN”在当前构建下不成立：KEK 可算 → 拿到 `pin_key` + `salt` → `verifier = KMAC(pin_key, slot_id||role||salt||pin)` 可离线穷举，4 位 PIN 是秒破的。

严重性：硬伤（就“密钥库机密性”这一属性而言，当前构建下它是 0）。

建议： 1. 把爆炸半径写准——不是“保护测试数据”，而是“在真实 KDR 落地之前，密钥库机密性与 PIN 防爆破均为零”； 2. 加一条 CI 断言：release 构建的二进制里不得出现 `KDR_STUB` 的字节序列。现有的 `tools/check_no_readback.py` 防的是硬件读回路，防不住这个常量根； 3. 换根的软件成本很低——`pqc_kdr_provider_t` 已经预留了缝（`kdr.c:37-41`），连 `hardware_backed` 标志位都在。这是设计上的先见之明，应尽快兑现。

H5 —— `API.md` 把愿景写成了现状

`docs/API.md:150-158` 的 PKCS#11 机制表标注：`CKM_AES_ECB / CKM_AES_CBC | sym_axi AES-128/256 | **Hardware**`，SM4/SM3 有 vendor-defined codes，`C_GenerateKeyPair → CMD_GENERATE → mlkem_axi KeyGen`。

实际情况（`src/p11/p11_module.c:586-589`）：机制表只有 4 个——`CKM_ML_DSA_KEY_PAIR_GEN`、`CKM_ML_DSA`、`CKM_ML_KEM_KEY_PAIR_GEN`、`CKM_ML_KEM`。整个模块没有 `C_Encrypt*` / `C_Decrypt*` / `C_Digest*`，没有任何 AES/SM4/SM3 机制，也没有 vendor code。调用链是 `C_GenerateKeyPair → hsm_slot_generate → pqc_backend_liboqs`（`p11_module.c:994`、`src/crypto/pqc.c:70`）——纯软件 liboqs，不经 `CMD_GENERATE`，更到不了 `mlkem_axi`。（公钥运算甚至绕过 `hsm_*` 直接调分发层：`C_Verify` 在 `:1522`、`C_Encapsulate` 在 `:1955`。两条路最终落到同一个后端。）

这里有一个必须纠正的常见误解（我自己起初也这么以为）：**不是**“加一句

`pqc_set_backend(pqc_backend_accel())` 就能走硬件”。`src/hal/pqc_accel.c:28-31` 的默认 transport 是软件桩（`if (!g_tr) g_tr = accel_transport_stub();`），而 `accel_stub.c` 内部（`:224/242/258/295/345`）全部回落到 `pqc_backend_liboqs()`；`axi` 与 `verilator` 两个真 transport 按 `include/pqchsm/accel.h:15-17` 只实现了 NTT 与 Keccak 两个算子，没有整算法的硬件路径。

也就是说：**PKCS#11 到硬件不是“一行配置”，而是一条尚不存在的路**。这把 H5 从“文档写错了”升级成“文档承诺了一条需要相当工作量才能建成的路”，也直接改变了 §5 中 P3-23 的代价估计。

另：`API.md:142` 称实现了 44 个 `C_` 函数，实数为 36 个。`API.md:5` 称“两个前端跑在同一套硬件上”——只有 SDF 前端碰过硬件。

严重性：硬伤（文档失真）。这条与 H1 同类：一份宣称“每一个数字都来自真实设备”的仓库，其对外 API 文档把未实现的能力标注成了 `Hardware`。这会直接误导集成方做技术选型。

建议： 把 PKCS#11 段改写为现状（4 个机制、纯软件后端），把硬件映射表移到“规划”小节并明确标注；修正函数计数。

H6 —— 吞吐表的数字在仓库里找不到日志，且与仓内日志差 2.8×

`README.md:136` 与 `TESTING.md:151` 发布：ML-KEM-512 @75 MHz = 924 / 1339 / 1018 ops/s。

`board/logs/` 全文搜索不到这三个数字。仓内唯一的 ML-KEM 性能日志 `RESULT_seckem3.txt:25` 是 329.8 / 521.7 / 389.9 ops/s ——慢约 2.8 倍，因为那次运行每一笔寄存器访问都经 SMC 由 EL3 代发（对照 `RESULT_service.txt`：一次端到端演示产生了 640 万次安全读 + 524 万次安全写）。

924 系列来自更早的普通世界 `/dev/mem` 直驱运行，其原始日志只存在于开源整理时删掉的内部文档里，从未以 `RESULT_*` 形式入库。

这与 `README.md:10`「本仓库中的每一个数字都来自真实设备」以及 `README.md:139-140`「原始板上抓取逐字保留在 `board/logs/`」直接冲突——对这张表而言，后一句不成立。`board/README.md:40` 的同类声明也不成立。

严重性：硬伤（对一个把”可溯源”当作核心卖点的项目而言）。数字大概率不是编的，但可溯源性断了，而且两种口径差 2.8 倍这件事 文档完全没提。

建议：三选——补回原始日志；或改用仓内有日志的数字并注明口径（EL3 代发）；或保留两组数字并解释差异来源。无论哪种，都必须说明“经 SMC 代发”与”/dev/mem 直驱”是两个不同的测量口径。

2.2 隐患

M1 —— 硬件熵源没有贯彻到密钥生成

代码在三处（`util.c:37-43`、`oqs_rng.c:46-59`、`hwrng.c` 文件头）反复强调”绝不静默回退软件源”。但真正生成密钥材料的地方全部直接调 `RAND_bytes`，根本不碰 `hwrng`：

用途	位置
私钥种子	<code>src/slot/slot.c:655</code>
备份根密钥 RMK	<code>src/backup/backup.c:86</code>
Shamir 多项式系数	<code>src/backup/shamir.c:114</code>
GCM nonce	<code>src/store/wrap.c:128</code>
KEK salt	<code>src/store/keystore.c:110</code>
pin_key / salt	<code>src/slot/slot.c:425, 469</code>

`pqc_random_bytes`（唯一 `hwrng` 优先且不回退的入口）只在 `src/hal/pqc_accel.c:121, 155` 被用到。而槽位管理器在 `install_key` 里自己生成种子后调 `pqc_keypair_from_seed`，绕过了后端的 `ac_keypair` —— 于是即便挂上硬件后端、即便板上 TRNG 完全正常，经槽位管理器生成的私钥，其种子熵仍来自 OpenSSL DRBG，不是 PL 环振噪声源。

这与项目的核心叙事（“真随机源在密码边界内”）正面冲突。风险场景：aarch64 上 OpenSSL DRBG 播种不当（容器内、早启动阶段 `getrandom` 未就绪），就是一个静默的弱密钥面。

严重性：隐患（当前 OpenSSL 播种正常时无害，但叙事与实现不符）。**建议：**把种子、RMK、salt、nonce 统一改走 `pqc_random_bytes`；至少在文档里明确”哪些随机来自硬件、哪些来自软件”。

M2 —— daemon 的句柄无会话隔离，且 dk 全程驻留

`service/pqchsm_fpgad.c:229` 的 `keys[MAX_KEYS]` 是进程全局。`OP_MLKEM_DECAPS (:358)` 只校验 `h < MAX_KEYS && keys[h].used`，不校验句柄归属哪个连接。而 `libsdfe` 的注释称”一个连接就是一个会话”。

失效场景：客户端 A 生成密钥（句柄 3，dk 留驻）→ 断开；客户端 B 连上，`DECAPS(a0=3, ...)` 直接使用 A 的私钥解封装。句柄空间只有 16，可枚举。0600 回环 socket 限制在同一用户，所以冲击有限，但与”会话”这个叙事矛盾。

同处还有一个不可达死代码：`keys_wipe()`（`:231`）的唯一调用点在 `:511`，而它位于 `for (;)`（`:486`，永不 break）之后。后果是 dk 在整个进程生命周期驻留内存，只有句柄槽位被复用时才被覆盖。

严重性：隐患。**建议：**句柄绑定到 accept 的连接；连接关闭时清该连接名下的 dk；把 `keys_wipe` 挪到信号处理或 `atexit`。

M3 —— `build-bl31-secmmio.sh` 的头部注释是从 `protread` 整段拷来的，把”读写”说成”只读”

`boot/atf/build-bl31-secmmio.sh:2-5` 与 `build-bl31-protread.sh:2-5` 逐字相同（只改了末尾引用的 .py 文件名），写着：「这一版比 BOOT0004 多的只有一个只读的保护单元读口（0x8200ff11）」。

而该脚本实际产出的是读 + 写的 secmmio 镜像——它自己在 :54 的符号自检就是 `for sym in pl_off_max pl_rd_ok **pl_wr_ok**`，产物是 `BOOT_SECMIO.BIN`。这个写口可达 key_vault 的 `KEY_IN` / `SLOT_CTRL` / `CTRL`（即装载密钥、erase、zeroize）。

一个安全敏感的构建脚本，头部宣称”只读”，实际向 PL 开了写口。构建脚本的头部注释是审计入口，读它的人会误判所建镜像的能力边界。

严重性：隐患（文档/构建一致性，安全敏感）。**建议：**改为”受限安全 MMIO 读写服务”，并列出入口可达的寄存器。

M4 —— SiP 不校验调用来源，边界完全等于 BL31 白名单

SiP handler 从不检查 `flags`（secure / non-secure）。这是有意设计（普通世界 Linux 就是调用方），但含义必须写清楚：**SiP 是一条安全事务隧道，它把 `AxPROT[1]=0` 的安全访问重新授予了普通世界。**

于是 PL 防火墙的 `SECURE_ONLY=1` 门控，对白名单内的寄存器不再对 root 提供保护；唯一的边界是 BL31 里的白名单。任何 EL1 代码（内核、加载的模块、内核漏洞）都能直接发这几个 SMC，`/dev/secmmio` 的 0600 权限只挡用户态进程、挡不住内核态。

具体残余能力（把”root 仍能驱动硬件”讲实）：经 PL_WR 到槽 1，root 可以 (a) 装载自己的密钥并用 sym 核当加密/MAC 预言机；(b) zeroize / erase / lock 密钥仓，对已配密钥做拒绝服务；(c) 提取现有密钥仍然不可能——无读出路径这一条经核实成立。

正面确认：白名单判据本身干净。`patch_atf_secmmio.py:114-143` 的 `pl_permit` 检查 4 字节对齐、`>= PL_BASE`、`rel < 6<<16`、`off <= pl_off_max[slot]`、逐槽读写权限，全是编译时常量表，无循环无指针算术；与 xbar `hit_of`（`axi4lite_xbar.v:147-162`）逐条比对，**SiP 白名单 $\subset$ xbar 译码 $\subset$ 防火墙窗口**。检查与访问在 handler 内原子，无 TOCTOU。

一个值得补的缺口：“白名单 $\subset$ xbar 译码”这个关键子集关系没有任何自动化检查，两处各自手写，只靠 `patch_atf_secmmio.py:24-27` 的注释声明。

严重性：隐患（送检口径已承认”MMIO 粒度”，此处是把残余能力显式化）。**建议：**文档把”边界 = BL31 白名单，不是 PL 防火墙”这句话前置；加一个脚本从 RTL 参数与 .py 常量表交叉生成/校验子集关系。

M5 —— TA 的 KDR 依赖 OP-TEE HUK，而 HUK 是否真绑硬件未经验证

`tee/ta/pqchsm_ta.c:44-83` 经 `PTA_SYSTEM_DERIVE_TA_UNIQUE_KEY` 取设备 xTA 唯一密钥。机制正确——前提是 HUK 真的来自硬件根（eFuse / PUF / CSU）。OP-TEE 在不少平台缺省使用一个测试常量 HUK，除非平台移植显式接了真实源。本仓库不含 OP-TEE core，无法证实。

若 HUK 是默认常量，则 KDR/KEK 可被任何持有相同 OP-TEE 构建的人离线推出，blob 跨设备可解、可伪造——H2 的预言机连设备都不用碰。

严重性：隐患（若确为默认 HUK 则升级为硬伤）。**建议：**上板验证——两块板对同一 salt 派生的 KEK 必须不同（比对 KDF 输出的杂凑即可，无需泄露密钥值）。这一项应在 JTAG 相关工作之前做，因为它决定了 §4 中 J3 的可行性。

M6 —— OP-TEE TZDRAM 的 XMPU 围护仍是草案，未上板验证

`tools/fsbl_xmpu/xmpu_protection_draft.c:1-14` 明标“尚未上板验证”，且注释指出 stock FSBL 有 `XFSBL_PROT_BYPASS`（DDR 保护被旁路）。

若 OP-TEE 的 TZDRAM (0x6000_0000) 未被 XMPU_DDR 标为 secure-only，普通世界经 `/dev/mem` 读该物理段可能直接拿到 KEK 与解包后的私钥——这会让 H2 的预言机变成多余（直接读内存即可）。`board/logs/` 里没有 TZDRAM region 的证据，只有 PL aperture 的反证。

严重性：隐患，潜在硬伤。建议：上板用 `xmpu_probe` / `prot dump group=3` 确认 TZDRAM 是否真被隔离。这是一次纯读操作，风险低，应尽快做。

M7 —— accel 后端用进程级 `static` 缓冲装私钥，且无锁

`src/hal/ppc_accel.c` 的 `ac_encaps_derand` (:138)、`ac_decaps` (:169)、`ac_sign` (:195)、`ac_verify` (:242) 都用 `static uint8_t buf[ACCEL_BUF_MAX]`，其中 decaps / sign 把私钥拷进这个进程级共享缓冲。

槽位管理器按每槽位加锁 (`slot_internal.h:65`)，所以两个线程在不同槽位上并发 decaps/sign 时，会同时读写同一个 `static buf` → 结果串扰、私钥跨操作泄漏。slot 层明确宣传线程安全（且有 `test_slot_concurrent`）。liboqs 后端用栈缓冲，无此问题；只有 accel 后端中招，当前被前端串行化（p11 全局锁、daemon 单连接串行）掩盖着。

附带：`ac_encaps_derand` 结束后未清 `buf`（decaps/sign 都清了），32 字节封装随机数 `m` 驻留在 static 缓冲里。

严重性：隐患（“没被现有接线触发的潜伏 bug”，但这正是最容易在改接线时爆的那类）。**建议：**改栈上缓冲或加后端级锁；补 `ac_encaps_derand` 的 `ppc_secure_zero`。

M8 —— `sym_axi` 的 PARAM0 实际落在 0x70，文档写 0x60，读 0x60 安静返回 0

`hardware/rtl/bus/sym_axi.v:280-293` 的读多路选择器先按 `f_araddr[6:2]` 分流 DOUT(12-15) / DIGEST(16-23)，落不到这两段才进 `case (f_araddr[5:2])`。我核过译码算术：

- 地址 0x60 → `[6:2] = 24`（越过 DIGEST 上界 23）→ 进 case，`[5:2] = 8` → 不匹配 `4'hC` → `default` → 返回 0；
- 地址 0x70 → `[6:2] = 28` → 进 case，`[5:2] = 12 = 0xC` → 返回 `0x0F`。

防火墙窗口是 `ADDR_MASK = 0x80` (`sym_axi.v:109`，即 0x00-0x7F)，两个地址都在窗口内，都不会 DECERR。而 `docs/REGISTERS.md:161` 写的是 `0x60 PARAM0 R 支持算法位图`。

失效场景：驱动按文档读 0x60 做上电自检，拿到 `0`，解读为“四个算法一个都不支持”；而真正返回 `0x0F` 的 0x70 是个未文档化的别名。又一个“安静给错值”。

严重性：隐患。建议：把 PARAM0 挪进 `[6:2]` 前置判断（如 `f_araddr[6:2] == 24`），让 0x60 与文档一致。

M9 —— `tamper` 在产品顶层恒接 0：一整套已验证的物理防护是死逻辑

`hardware/rtl/board/zu3eg_hsm_top.v:272`：`wire tamper = 1'b0;`

`tamper` 扇出到 `trng` / `key_vault` / `sym` / `mlkem` 全部四个从机，是所有 `tamper_latched`、密钥仓一拍全清、防火墙 fail-closed 的唯一硬件触发源。恒零意味着这些防护逻辑在实际下载的位流里全部是死逻辑，只有软件

`ZEROIZE` 路径活着。板上日志也如实记录了这一点（`RESULT_audit.txt` 的 SKIP：「这一版 bitstream 里 tamper 恒接 0，板上没有任何东西能拉起它」）。

值得单独指出的是，这些 tamper 相关逻辑质量很高：`key_vault.v:101` 与 `axi4lite_firewall.v:142` 都用 `tamper || tamper_latched`（组合项在前），正确消掉了“锁存下一拍才生效”的一拍 TOCTOU 窗口。写得很对的防护，却从未在真硅上通过电。

严重性：隐患（项目已声明，但“已验证的死逻辑”这个状态值得单独点名）。**建议：**见 §4 的 J4——这是一个代价极小、收益明确的改动，且不需要 JTAG。

2.3 其它值得记录的问题（优化级）

#	问题	位置
L1	<code>hsm_slot_force_state</code> 是绕过 ACL 的测试后门，没有 <code>#ifdef</code> 隔离，编译进生产库。不可远程触发，但应包起来	<code>src/slot/slot.c:387</code>
L2	构建加固缺失：无 <code>-D_FORTIFY_SOURCE=2</code> 、无 <code>-fstack-protector-strong</code> 、无 <code>-Werror</code> ；fuzz 目标未见强制 ASan/UBSan，而 <code>CMakeLists:165</code> 的注释声称跑在 ASan 下	<code>CMakeLists.txt</code>
L3	daemon socket 在 <code>/tmp</code> （世界可写、可被 squat）。但 <code>chmod 0600</code> 先于 <code>listen</code> 这个顺序是对的，竞态窗口已关闭	<code>service/pqchsm_fpgad.c:473-483</code>
L4	审计链是无密钥哈希链，两次锚点之间可被有写权限者整体改写成自治；而没有任何东西调度锚点，频率全靠运维自觉	<code>src/audit/audit.c:150</code> ；锚点 <code>anchor.c:267</code>
L5	<code>check_zeroize.py</code> 不扫 <code>service/</code> ，所以 daemon 的 <code>keys[]</code> / <code>seed[64]</code> / <code>buf[4900]</code> 不在其视野内（M2 的死代码正是因此没人管）	<code>tools/check_zeroize.py</code>
L6	<code>check_no_readback.py</code> 只对 <code>kdr.h</code> 一个头文件做 8 个子串黑名单匹配，却被 <code>security-policy.md</code> §3 引用为“结构性地保证所有接口不返回私钥”	<code>tools/check_no_readback.py</code>

#	问题	位置
L7	<code>encaps.v</code> 的 <code>obcnt</code> 8 位回绕 经复核为非问题：真正堵住这条路的是 <code>:703</code> 的每段清零（不只是 <code>cnt</code> 控制 <code>u</code> 段边界），<code>v</code> 段不会被 <code>u</code> 段残值污染。仅作参考	<code>hardware/rtl/mlkem/encaps.v:703</code>
L8	<code>trng_health</code> 的 RCT 用 <code>>=</code> <code>RCT_C</code>、APT 用 <code>> APT_C</code>。两个 cutoff 都按“使 $P \leq \alpha$ 的最小整数”选出，与之配套的判据应是 <code>>=</code>；写成 <code>></code> 等于把判据挪到 <code>C+1</code>，APT 实际虚警率是预算的 1/2.15（RCT 是 1.00 倍）。方向是保守的（少报警），量级小，不是安全漏洞。真正值得记的是：参考模型 <code>trng_health_model.py:84</code> 用了同一个 <code>></code>，所以 RTL↔模型逐样本对拍在结构上不可能发现它——要卡必须拿 $P(X \geq C) \leq \alpha$ 的定义式本身去卡	<code>trng_health.v:113, 128</code> ；模型 <code>:72, :84</code>
L9	<code>mlkem_axi</code> 不校验软件实际写入字节数，欠填会喂入缓冲残留并正常完成（DONE 置位、OUT_LEN 有值）。KeyGen 那条比 Encaps 更值得担心：<code>:479-491</code> 的 <code>S_PRE</code> 无条件读 64 字节，软件只写 32 字节 <code>d</code> 时 <code>z</code> 取残留，冷启动/zeroize 后残留是全 0 → 隐式拒绝密钥 <code>z=0</code> 可预测，而 <code>ek/dk</code> 一切正常，KAT 之外无从发现。当前在树的调用方都自律地校验了 <code>MK_INPTR</code>（<code>pqchsm_fpgad.c:202</code>、<code>hsm_kem3.c:164</code>），所以无不可信主体能触发；但硬件侧没挡、cocotb 也没有短喂用例	<code>mlkem_axi.v:210-213, 479-491, 515-521</code>

#	问题	位置
L10	<code>ooc_seq.xdc:18</code> 的 <code>remove_from_collection</code> 在 XDC 上下文不受支持，该行 <code>set_input_delay</code> 失效、in2reg 的建立/保持分析丢失。修正两点：一是 Vivado 会打带行号的 CRITICAL WARNING，不是无痕，只是非致命、退出码 0，容易在长日志里漏掉；二是不传导到最终比特流—— <code>impl_bitstream.tcl:89</code> 只读 <code>board_pins.xdc</code>，整机对外只有 fan 一根脚且已 false_path，模块间接口在整机里是 reg2reg 会被正常计时。受损的是 OOC 时序报告的可信度，不是硅上行为	<code>hardware/syn/constraints/ooc_seq.xdc:18</code>
L11	<code>security-policy.md</code> 是软件时代旧稿局部翻新，内部至少四处自相矛盾（§9 “模块无熵源” vs §13.2 “已闭合”；§1 “单芯片硬件模块” vs §7 “就目前的软件模块而言不适用”；GM/T 段”SM3/SM4 均未实现” vs 全库跑通 GB/T 向量）	<code>docs/reference/security-policy.md:18/127/148/244/289</code>
L12	汇总口径经不起逐行数：「20/20 byte-exact」实为 18 条 ACVP + 1 条常数时间 + 1 条性能测试跑完就 <code>n_pass++</code>（<code>hsm_kem3.c:310</code>，无任何比对）；「6/6 DECERR」实为 5 项被拒 + 1 项对照读成功	<code>README.md:130</code> 、 <code>TESTING.md:24,32</code>
L13	计数类不一致：ctest 46 vs 45（CONTRIBUTING、deployment）；“26 top levels” vs 实际 32；“八条豁免” vs 工具输出 9 条；<code>impl_bitstream.tcl</code> 内部 +0.010 vs +0.011	多处
L14	悬空引用（开源整理漏网）：<code>tools/amdahl.py:12</code>、<code>tools/cycle_budget.py:68</code>、<code>tools/fetch_vectors.sh:5</code>、<code>src/p11/p11_module.c:29,1718</code> 指向的 <code>docs/</code> 文件均不存在	多处
L15	<code>TESTING.md:34</code> 的 sanitizer 一行指向 <code>USAGE.md</code>，而 <code>USAGE.md</code> 里没有任何 sanitizer 内容；CMakeLists 与 CI 均无 sanitizer job	<code>docs/TESTING.md:34</code>

2.4 测试盲区

1. **service/** 层零自动化测试。会话/句柄/串行化是该层存在的理由，但 **tests/** 里没有一个 service 测试，CI 只 `make -C service` 编译。唯一证据是 `RESULT_service.txt` 一次单客户端 happy-path。并发、句柄失效、错误路径全部未测——**M2 就活在这个盲区里**。
2. **zu3eg_hsm_top** 整机集成从未仿真。cocotb 明确不含它；xbar、firewall、各从机都是单独对拍。历史提交 `5da60c0`（“上板复验：全绿，但先红了六条”）就是这个缺口的代价记录。
3. **fan_ctrl_axi**（槽 5）没有任何 cocotb 测试，也没有寄存器表。`REGISTERS.md:5-8`「每一条款由 cocotb 测例单独验证」对槽 5 不成立。
4. **board/kmod** 三个内核模块、**boot/atf** 四个补丁脚本：无自动化测试。
5. 写路径的总线拒绝在板上永远测不了（SError → panic），WIPING 期间 SLVERR、tamper 同拍语义均为仿真单证。
6. CI 无 sanitizer、无 fuzz、无 aarch64 job。
7. “EL3 白名单 ⊆ xbar 译码”无自动化检查（见 M4）。

三、可优化点

3.1 性能：结论是”赢面在时钟不在架构”，且这个结论是可信的

先把口径说清楚——4.33x 与 93.6 MHz 都不在本分支上，它们属于 `zu3eg-fpga-perf` 分支（`c835f02`）。本分支（`96f9b55`）的时序数字是 整机 WNS +3.504 ns @ 75 MHz。

`zu3eg-fpga-perf` 的测量口径相当严谨，值得肯定：A53 单核 @ 实测 1066.67 MHz（读 `scaling_cur_freq`，不用标称值）、liboqs 0.16.0 且运行时确认 NEON 优化实现生效、用去随机化 API 先逐字节对上 ACVP 向量再计时（避免“没跑完整算法的软件基线”偏快）。九项全部逐字节一致后才比较——比的是同一件事。

结果：

	软件 (μs)	FPGA-PIO	FPGA-DMA	DMA vs 软件
九项合计	1422	10612	6164	0.23x

关键在于拆解，而不是这个倍数本身：

- FPGA 的”拍效率”是真的——同一次运算核只花 21k–84k 拍，CPU 要 87k–275k 周期，硬件每一步做的事确实多 3.3–4.5 倍；
- 但时钟差 14.2 倍（75 MHz vs 1066.67 MHz）。要赢，拍效率得 > 14.2x。

把搬运开销全部归零、只算纯核时间，仍比软件慢 3.3–4.5 倍。所以这不是接口问题——PIO → DMA 那 2.08x 是真的，但它填的是一个 14 倍的坑。

要真的赢，核拍数得再压到 1/3.7（768 KeyGen 从 36615 拍压到 9855 拍以下）。而 profile 是 XOF 22–41%、MAC 16–33%、NTT 21–28%——没有单独一块能给出 3.7x（NTT 这一轮已经从 3457 压到 1177 拍了）。要是整体重做：多路并行 Keccak + 多蝶形 + 更宽数据通路 + 提频。那是另一个量级的工作。

我的评估：这个结论是可信的，方法论无懈可击，而且项目自己给出了正确的解读（“要不要继续追，是产品决策不是工程决策”）。

优化建议：不建议继续追吞吐。理由不是做不到，而是投入产出比不对——在一块 ZU3EG 上要赢过带 NEON 的 A53，需要重做整个数据通路，而这条路的价值本来就不在吞吐。如果一定要动，优先级应当是：提频（见 3.3）> 并行 Keccak（占比最大且最规整）> 其它。

3.2 资源：约 1566 行 RTL 在产品形态下不可达

死逻辑	规模	原因
<code>pqc_accel_axi.v</code> + 专用 <code>axi4lite_regs.v</code>	~911 行	未被 <code>zu3eg_hsm_top.v</code> 例化；板级顶层无 AXI4-Stream 主口、无 DMA
整个 <code>hardware/rtl/mldsa/</code>	~655 行	在 <code>bus/</code> 、 <code>board/</code> 下零引用，只被 <code>tb_mldsa_units.v</code> 驱动。是一堆算子，不是一个核

经独立复核，行数与“零引用”均属实，但“死逻辑”这个框架需要收紧三点：

- 不是“未参与构建”，而是“被解析、被剪枝、占零资源”。`impl_bitstream.tcl:66` 的 `foreach d {common mlkem mldsa keccak trng sym bus board}` 把 `mldsa/` 与 `bus/` 都读进了产品综合，只是 `synth_design -top zu3eg_hsm_top` 在细化阶段把未被引用的层次剪掉了。
- 是“未上板”，不是“未验证”。这些模块各自作为独立 top 进了 OOC 综合与 lint（`ooc_synth.tcl:44` 显式列了 `mldsa_ntt_core` 与 `pqc_accel_axi`），并有 27 个 cocotb 用例（`tb_mldsa_units` 6 + `mldsa_ntt_core` 5 + `pqc_accel_axi` 16）。
- 这不是缺陷，是被明确记录的作用域决策，共五处披露（`zu3eg_hsm_top.v:70-77`、`ARCHITECTURE.md:128`、`SECURITY.md:157`、`API.md:155` 等），且顶层注释主动写明“mlkem 对 `ntt_core/sha3_core` 只是间接覆盖，不是独立 KAT”。

所以这一节的意义不在“指出问题”，而在**量化**：审计与资源核算时应当把这 1566 行剔除，且应给它们一个明确的状态标签（见 §3.4 建议 3），避免变成没人记得的半成品。

另有一处 `zu3eg-fpga-perf` 分支上的**真死逻辑**，值得单独讲，因为它是一个很好的教训：

BL31 白名单 `patch_atf_secmmio.py` 是 `PL_NSL0T = 6`（只到槽 5），而 **DMA 在槽 6**。于是产品形态下 `pl_permit()` 里 `rel >= (6<<16)` 直接返回 0——普通世界碰不到它（对的），**安全世界也碰不到**。结果是：**这个引擎谁都启动不了，544 LUT + 2 RAMB18 的死逻辑**。那 2.08x 是在 open bitstream 上量的，产品形态那条 EL3 路径至今仍是逐字节 SMC。

这条项目自己发现并写进了提交说明，处理端正。要闭合需三步：白名单 6→7、重建 BL31 换 BOOT.BIN、服务层改成先编程 DMA 再起核。但鉴于 3.1 的结论（这条路比软件慢 4.3 倍），我同意“先不接进生产”的判断。建议把它明确标注为“已验证但未接线的实验特性”，避免它变成没人记得的半成品。

资源现状（本分支，`xazu3eg-sfvc784-1-i` 完整布局布线）：LUT 35,611 / 70,560 (50.47%)、寄存器 25,916 (18.36%)、BRAM 15.5 / 216 (7.18%)、DSP 140 / 360 (38.89%)。还有一半器件空着，所以资源不是当前的约束——这也意味着“加 SM2 核”或“把 ML-DSA 串成核”在资源上是可行的。

3.3 时序：余量是被管理的，但提频的路比看上去远

本分支：整机 WNS +3.504 ns @ 75 MHz，有效保持余量 0.110 ns。 `mlkem_decaps` OOC 收敛 108.5 MHz（选 75 MHz 留 45% 余量的依据）。 `pqc_accel_axi` OOC -0.047 ns / Fmax 99.5 MHz（`hardware/syn/README.md:95-96`，坦承未修）。

perf 分支的 Fmax 演化：103.2（改造前）→ 95.3（加流水 NTT + DMA）→ 93.6（并入 crypto 线的保持修复流程后）。注意方向——加了优化之后 Fmax 反而降了，这是保持时序修复用建立余量换来的，方向是对的，但说明提频空间在收窄。

RTL 里最可疑的两条长组合路径（来自评审，我认为判断合理）：

1. `keccak_f1600` 单轮全展开（`hardware/rtl/keccak/keccak_f1600.v:100-128`）： θ 的列奇偶 + D 扩散 + ρ/π 的 25 路变量循环左移 + χ + ι 全在一个 `always @(*)` 里，每拍推进一轮，1600 位状态一次过。这是 SHA-3 天然的 Fmax 天花板，将来提频这里要先切（ χ 之后或 θ 之后打一拍）。
2. AES 解密方向的 `inv_mix_cols`（`aes_core.v:216-237`）：每列调 4 次 `gm()`，而 `gm()` 是 8 次迭代的 $GF(2^8)$ 乘法，16 个 `gm` 展开是相当深的异或树，且串在 `InvSubBytes` 之后。正向 `mix_cols` 只用 `xt` ($\times 2/\times 3$) 浅得多。建议在 OOC 报告里单独看 `aes_core` 解密方向的 WNS。

提频到 100 MHz 的障碍不止于逻辑：`pl_clk0 = 150 MHz`，`BUFGCE_DIV` 只能整数分频，拿不到 100 MHz——需要加 MMCM。这是一个常被忽略的实现约束。

约束质量评估（正面）：`trng_ro.xdc` 只留纯声明式约束，把需要 `if` 的部分正确地放进 `tcl`；环振的 `set_false_path` + `ASYNC_REG` 两级同步器处理妥当；`trng_ooc.tcl:71-102` 对“环振是否被综合优化掉”做了综合后 + 布线后双重计数检查（环被约掉会静默退化成常数发生器，这个检查是必需的）。唯一存疑是 `L10` 那条 `ooc_seq.xdc:18`。

3.4 架构

做得对、建议保持的：- 分层清晰（算子 → 核 → 总线 → 板级），“同一段数学只有一份实现”这条纪律贯彻得好（`barrett/mont/butterfly/bmzeta` 都是单一来源），从根上消除了“两份实现改一处忘另一处”的漂移；- 边界哲学（“没有路径”而非“门控”）正确且被 RTL 结构落实；- 双仿真器交叉（Verilator 2-state / Icarus 4-state，宽度截断语义差异本身是一重交叉检查）。

建议改的：

1. `SECURE_ONLY` 应当是构建参数，不是源码改动。现状（改过去回不来）直接导致了 H1。应提供 `dev` / `certification` 两个配置，由 `impl_bitstream.tcl` 选择，文档同时描述两者。
2. 主机栈与硬件路径应当合流，或明确分家。现在有两条互不相干的路：`service/`（SDF，真碰硬件）与 `src/ + p11`（软件 `liboqs`，从未碰硬件）。`ARCHITECTURE.md:234-238` 承认了这一点，但 `API.md` 又把它们描述成“两个前端跑在同一套硬件上”（H5）。要么让 `p11` 走 HAL 到硬件，要么在文档里彻底分家。当前的中间状态是失真的来源。
3. `axi4lite_regs.v` 与 `pqc_accel_axi.v` 应明确标注状态。要么接线，要么移到 `experimental/`。

四、安全收口：假设 JTAG 已到位

前提：JTAG 今晚到位，因此所有 JTAG 相关项都按”可做”来规划，不再是受阻项。

4.1 先纠正一个前提：BBRAM 不等于信任根

这一条必须先讲清楚，否则整个方案会建立在错误前提上。在 ZynqMP 上：

- BBRAM 存的是 AES-256 加密密钥，电池供电、可反复擦写、JTAG 可编程。对单板原型是可逆的——这正是它比 eFUSE 适合当前阶段的原因。
- 认证根（RSA）在 eFUSE：PPK hash + RSA_EN。不烧 eFUSE，BootROM 就不强制认证——攻击者换一份”声明为不加密/不认证”的镜像，BootROM 照启不误。
- 同理 ENC_ONLY 不烧，BootROM 也不强制加密启动。

所以：BBRAM-only 提供的是启动镜像的机密性与完整性（AES-GCM），不是防替换。真正的”防替换”必须烧 eFUSE，而 eFUSE 不可逆 + 只有一块板 = 本项目已明确排除。

这个区分必须写进 SECURITY.md。做完 BBRAM 方案后如果宣称”已获得硬件信任根”，那就是一次新的夸大，性质与 H5、H6 相同。准确的表述是：把”root 换一份 bitstream 就换掉整个边界”降级为”需要物理接触且要能重打镜像”。

4.2 事实基线（已核实）

事实	出处
BOOT.BIN 用 bootgen -w 打，无 -authentication / -encrypt	build-bl31-seccmmio.sh:76
bif 里已有 [destination_device=pl] 一行，但指向 pl_fanquiet.bit（风扇设计）	同上 :68
密码 bitstream 是运行时 fpgautil -b ... -f Full 从 SD 卡加载	board/scripts/plharness.sh:66-67
仓库内无任何 BBRAM / 签名 / eFUSE 工装	全树 grep 无带 auth 的 .bif、无 program_bbram
tamper 顶层硬接地	zu3eg_hsm_top.v:272
SYSMONE4 的 ALM / OT 输出空接	hardware/platform/fan_ctrl/fan_sysmon.v:99-100
KDR provider 接口已预留 hardware_backed 位	src/crypto/kdr.c:37-41
multiboot 由 POR 清零 → 坏 BOOT.BIN 无法靠断电救回	board/factory/make_boot_scr.sh 文件头

4.3 关键判断：JTAG 的头号价值是”可恢复”，不是 BBRAM 本身

当前启动侧的所有加固都被同一个约束卡死：板子没有 JTAG，写坏 BOOT.BIN 就是砖（multiboot 由 POR 清零，非 golden 槽根本轮不到）。这个约束的后果不是”少了一个功能”，而是整条启动链的改动都被禁止了——所以边界才退到”运行时 fpgautil 装 bitstream”这种任何 root 都能替换的形态。

JTAG 到手后先变的是风险计算，不是能力清单。因此正确的顺序是：先建立并实测一次真实的救砖流程，再动任何一项加固。否则第一次刷失败就又回到今天的处境。

4.4 方案（按依赖顺序）

J0 — 建立恢复能力（前置，不加固任何东西）

用 JTAG 走一遍完整的”故意刷坏 → JTAG 救回”演练，把步骤固化成脚本。基线用已知 10/10 能启的 `BOOT_OPTEE_FIXQ.BIN`。

这一步不产出任何安全属性，但它是后面每一步的许可证。代价：半天。没有这一步，J1–J3 都不该开工。

J1 — PL 固化进黄金 BOOT.BIN 【最高性价比】

bif 里 `[destination_device=pl]` 从 `pl_fanquiet.bit` 换成密码 bitstream。

收益：PL 边界（firewall / key_vault / SECURE_ONLY）由 FSBL 从受控镜像加载，不再是 SD 卡上一个 root 可替换的 `.bit` 文件。这消掉了一整类“换 bitstream 即换掉整个密码边界”的攻击——而这正是 `SECURITY.md:60–61` 自己列为”不防”的第二条。

代价：一行 bif 改动 + 一次刷写。管道早就通了，只是指错了文件；唯一拦路的是刷坏救不回来，而这正是 J0 消除的。

注意：风扇温控目前正是靠那份 bitstream，合并 PL 设计时别把风扇丢了（否则重演”开机满速吵”）。两个设计已在同一 bitstream 里共存过，有先例。

J2 — BBRAM 加密启动

`program_bbram` 经 JTAG 写入 AES-256；bif 加 `keysrc_encryption=bbram_red_key`，对 FSBL / PMUFW / ATF / OP-TEE / bitstream 分区加密。

收益：镜像机密性 + GCM 完整性。拔卡读不出 BL31 白名单与 OP-TEE 内容。限度：不防替换（见 4.1），必须在文档里写明。

代价：中等。需要新建一套带密钥的构建流程，且密钥管理本身成为新问题（BBRAM 密钥存哪、怎么备份、电池没电怎么办——电池失效 = 板子起不来，这一点要在部署文档里写清）。

J3 — 设备绑定密钥根，替换 KDR 桩

软件侧成本很低：provider 接口已预留（`kdr.c:37–41`），换 provider 即可。

但有一个必须先回答的问题：BBRAM 的 AES 密钥软件读不出来（CSU 专用），所以不能直接拿来当 KDR 值。可行路径两条：

- (a) Device DNA（57 bit，可读）——历史提交 `62c9aac` 已走过这条。但 DNA 是公开可读的，只能做设备绑定，不能当秘密。用它派生 KDR，等于”任何知道这块板 DNA 的人都能复算 KDR”——比常量桩强（跨设备不通用），但挡不住拿到过设备的人。
- (b) OP-TEE HUK ——这才是真秘密根，但 HUK 是否真绑硬件尚未验证（M5）。

诚实的结论：JTAG 到位后 KDR 能从”编译期常量”升到”设备绑定”，但要升到”攻击者拿到固件也推不出根”，取决于 HUK 的真实性，这一条不是 JTAG 能解决的。所以 M5 的验证必须排在 J3 之前。

J4 — 接上 tamper 源【不需要 JTAG，应当并行做掉】

`tamper` 硬接地导致 key_vault 一拍全清、firewall fail-closed、TOCTOU 消窗这一整套已验证过的 RTL 全是死逻辑（M9）。而同一片 fabric 里，SYSMONE4 的 `ALM` / `OT` 输出正空接着（`fan_sysmon.v:99–100`）——现

成的环境篡改源就在旁边。

接上即可让这套逻辑在真硅上活过来。另一个候选源是 `trng_alarm`（顶层已有该信号）。

必须写明的风险： – 误触发会清空密钥仓，所以要定阈值与迟滞； – 这给了攻击者一条”烤板子即 DoS”的路。这是**有意的权衡**（密码机本应当宁清勿泄），但要写进文档而不是留给使用者去发现； – 这会跨越 README 声明的”风扇逻辑与密码逻辑无共享信号”这条边界，是一个需要显式记录的架构决策。

代价：很小。收益是把一大块高质量但从未通电的防护逻辑激活，并让 `RESULT_audit.txt` 里那条 SKIP 变成可测项。

4.5 JTAG 到位后仍然做不到的

为免方案被误读为”全部收口”，明确列出：

- 防镜像替换（需 eFUSE `RSA_EN` / `ENC_ONLY`，已因单板不可逆而排除）；
- PUF 黑密钥（依赖 eFUSE）；
- 功耗 / EM 侧信道（需要侧信道试验台，项目已明确排除，理由充分）；
- 操作粒度 SiP（M4 的根治方案，需要 EL3 里跑状态机，是重构范畴）。

五、优先级路线图

排序原则：先修会伤到人的，再修能被远程/本地触发的，最后做需要新硬件的。

P0 — 立刻做（数小时到一天，风险极低）

#	事项	理由	代价
1	H1: 把 SError 警告提进 README/REGISTERS, 修正三处 <code>SECURE_ONLY</code> 描述	唯一一条会直接伤到使用者 的问题（照文档操作 → panic → 断电）	1 小时
2	H2: TA 私钥 KEK 域分隔	硬伤，普通世界应用即可触 发，改动小	半天
3	H5 / H6: 修正 <code>API.md</code> 的 PKCS#11 段与吞吐表口径	对外失真，影响集成方选型 与项目可信度	半天
4	H4: 把 KDR 爆炸半径写准 + 加二进制常量 CI 断言	声明成本极低，误导成本很 高	2 小时
5	M5: 上板验证 HUK 是否真 绑硬件	纯读操作，且它决定 J3 是 否可行	半天
6	M6: 上板确认 TZDRAM 是 否真被 XMPU 隔离	纯读操作；若未隔离则 H2 之外还有更直接的泄漏路径	半天

P1 — 本周内 (JTAG 相关的前两步 + 明确的隐患)

#	事项	理由	代价
7	J0: JTAG 救砖流程演练并脚本化	后面所有启动侧工作的许可证	半天
8	J1: PL 固化进黄金 BOOT.BIN	本轮性价比最高的安全收口: 一行 bif, 消掉”换 bitstream 即换边界”	半天 (含刷写验证)
9	J4: 接上 tamper 源 (SYSMON ALM/OT)	不需 JTAG, 激活一大块已验证的死逻辑	1 天 (含阈值/迟滞与回归)
10	M2: daemon 句柄绑定连接 + 修 <code>keys_wipe</code> 死代码	会话叙事与实现不符, 且 dk 长期驻留	半天
11	M3: 修 <code>build-bl31-secmmio.sh</code> 头部注释	安全敏感的构建脚本宣称”只读”实为读写	15 分钟
12	M8: <code>sym_axi</code> PARAM0 地址 + L9: <code>mlkem_axi</code> 欠填校验	两处”安静给错值”, 是最贵的一类失败	半天

P2 — 本月 (结构性问题)

#	事项	理由	代价
13	H3: keystore 防回滚 epoch	硬伤, 但需要先有防回滚存储, 故排在 J2/J3 之后	2—3 天
14	J2 + J3: BBRAM 加密启动 + 设备绑定 KDR	依赖 J0/J1 与 M5 的结论	3—5 天
15	M1: 随机源统一走 <code>pqc_random_bytes</code>	消除”叙事说硬件、实现走软件”的矛盾	1—2 天
16	M7: accel 后端 static 缓冲改栈上/加锁	上板并发驱动前必须清	半天
17	<code>SECURE_ONLY</code> 改为构建参数 (架构建议 1)	从根上防止 H1 复发	1 天
18	补 <code>service/</code> 层测试 + 把 <code>service/</code> 纳入 <code>check_zeroize.py</code>	当前最大的测试盲区, M2 就活在这里	2 天
19	L11: 重写 <code>security-policy.md</code>	四处自相矛盾, 作为送检草案不可用	1 天

P3 — 视产品决策而定

#	事项	判断
20	追吞吐（并行 Keccak / 更宽通路 / 提频 + MMCM）	不建议 。需重做数据通路才能赢过带 NEON 的 A53，而这条路的价值不在吞吐（§3.1）
21	把 ML-DSA 串成完整核	资源够（还空一半）。若签名是产品需求则值得做，否则 655 行算子应标注为实验特性
22	加 SM2 核	若面向 GM/T 送检则 必须 做（ <code>security-policy.md</code> 自己指出 GM/T 0028 的算法列表要求 SM2）
23	让 PKCS#11 走 HAL 到硬件，或与 <code>src/</code> 彻底分家	决定 <code>API.md</code> 该怎么写。 代价被普遍低估 ：不是换个后端，而是要新建整算法的硬件 transport（现有 axi/verilator 只实现 NTT 与 Keccak 两个算子， <code>accel_stub</code> 全部回落 <code>liboqs</code> ）。 建议先分家、把文档改准，硬件路径作为独立立项 （见 H5）
24	操作粒度 SiP（根治 M4）	重构范畴，EL3 无异常处理，需要独立的验证方法
25	DMA 接线（白名单 6→7）	鉴于 §3.1 结论， 建议保持不接 ，但明确标注为”已验证未接线”

附注：关于本报告的置信度

- 我亲自逐行复核并确认的：H1（`SECURE_ONLY` 倒挂与 SError 警告缺失）、H2（TA 预言机的完整链条）、H3（keystore 无 epoch）、H4（KDR 桩与派生链）、H6（吞吐数字无日志背书）、M1（熵源绕过）、M2（句柄与死代码）、M3（构建脚本注释失配）、M8（PARAM0 译码算术）、M9（tamper 恒零）、以及 §4.2 的全部事实基线。
- 来自专项评审、我做了交叉印证但未逐行复核的：M4、M5、M6、M7、L 系列的大部分、§2.4 测试盲区。
- 经一组以”证伪”为默认立场的独立验证复核过的六条（结果已回写进正文）：

断言	裁决	复核带来的修正
<code>mlkem_axi</code> 欠填校验（L9）	成立·隐患	发现了更严重的一站：KeyGen 的 <code>S_PRE</code> 会让 <code>z</code> 取全 0 残留，隐式拒绝密钥可预测
TRNG 的 APT 边界（L8）	成立·隐患	定级压一档：方向保守、量级小；但参考模型有同一处错，对拍结构上抓不到

断言	裁决	复核带来的修正
<code>ooc_seq.xdc</code> 约束失效 (L10)	成立·隐患	是 CRITICAL WARNING 不是无痕； 不传导到最终比特流 ，损的是 OOC 报告可信度
死逻辑 1566 行 (§3.2)	成立·优化	不是缺陷 ，是有五处披露的作用域决策；且“未上板”≠“未验证”（有 OOC + lint + 27 个 cocotb 用例）
<code>encaps.v</code> 的 <code>obcnt</code> 回绕 (L7)	成立·非问题	真正的保护是 <code>:703</code> 的每段清零；已降级为记录
PKCS#11 走软件后端 (H5)	部分成立	推翻了我的一个前提 ：硬件路径不是“一行配置”，而是尚不存在——已据此改写 H5 与 P3–23

- 无法在本次只读评审中验证的：M5（HUK 真实性）与 M6（TZDRAM 隔离）都需要上板实测，已列入 P0。

最后一句评价：这个项目的**问题不在于它做错了什么难的事，而在于它做对的那些事没有被文档如实地反映出来，以及”最后一公里”（信任根、密钥全生命周期、文档与代码同步）还没有走完。**前者是可以在一两天内大幅改善的；后者是接下来一个月的主线。